

week10 개인 요약본

≡ 다중 선택

01 차원 축소 개요

• 차원 축소란?

- 매우 많은 피처로 구성된 다차원 데이터 세트의 차원을 줄여 새로운 차원의 데이터 세트를 생성하는 것
- 고차원 데이터의 문제점 (= 차원의 저주)
 - 데이터 포인트 간의 거리가 기하급수적으로 멀어지고, 희소(sparse) 구조를 가짐
 - 적은 차원에서 학습된 모델보다 예측 신뢰도가 떨어짐
 - 피처 간 상관관계가 높아질 가능성 증가 → 선형 모델에서 다중 공선성 문제 발생

• 차원 축소의 두 가지 방식

- **피처 선택(Feature Selection)**: 불필요한 피처를 제거하거나 중요한 피처만 선택
- **피처 추출(Feature Extraction)**: 기존 피처를 단순 압축하는 것이 아니라, 피처를 더 잘 설명하는 또 다른 공간으로 매핑하여 저차원의 중요 피처로 압축해 추출 → 새롭게 추출된 특성은 기존 피처와 완전히 다른 값이 됨

• 대표 알고리즘: PCA, LDA, SVD, NMF

- 이미지 데이터: 잠재적 특성을 피처로 도출해 이미지 변환 및 압축
- 텍스트 데이터: 단어 구성에서 숨겨진 시맨틱(Semantic) 의미나 토픽(Topic)을 잠재 요소로 추출 → SVD, NMF가 기반 알고리즘으로 사용됨

• 매니폴드 학습

- d차원 매니폴드: n차원 공간의 일부로, d차원 초평면처럼 보이는 구조 ($d < n$)
- 대부분의 고차원 데이터는 저차원 매니폴드에 가깝게 놓여 있다는 가정(매니폴드 가정)에 기반
- 단순 투영이 아닌, 매니폴드를 "펼쳐서" 저차원으로 변환해야 구조를 보존할 수 있음 (예: 스위스롤 데이터셋)

02 PCA (Principal Component Analysis)

• 기본 아이디어

- 변수 간의 상관관계를 이용하여 주성분(Principal Component)을 추출해 차원을 축소 + 정보 유실 최소화가 목표 → 가장 **높은 분산(=데이터의 변동성)을 가지는 데이터의 축(→ PCA의 주성분)**을 기준으로 데이터를 투영해 차원 축소
- 주성분 구하기
 - 첫 번째 주성분 (PC1): 데이터가 가장 많이 퍼진 방향 (최대 분산)
 - 두 번째 주성분 (PC2): PC1과 직교하면서 그 다음으로 분산이 큰 방향
 - n+1번째 주성분 (PCn+1): PCn과 직교하면서 그 다음으로 분산이 큰 방향
 - → "벡터 축의 개수 = 차원"
- 작은 수의 주성분으로 원본 데이터의 총 변동성을 대부분 설명 가능

• 선형대수 관점

- 핵심: 공분산 행렬을 고유값 분해해서 고유벡터를 새로운 축으로 사용
- 공분산 행렬은 모든 피쳐 간의 관계(분산+공분산)를 담은 전체 정보이며, 이를 고유값 분해하면 특정 방향(고유벡터)에서의 분산 크기(고유값)를 얻을 수 있음
- **공분산 행렬(Σ)의 고유값 분해: $\Sigma = Q\Lambda Q^T$**
 - Q: 고유벡터 행렬 (새 좌표축) → 직교 행렬 (대칭 행렬이므로 항상 직교 고유벡터 보장)
 - Λ : 고유값을 모은 대각행렬 (각 축의 분산 크기, 큰 순서대로 정렬)
 - 고유벡터: PCA의 주성분 벡터, 입력 데이터의 분산이 가장 큰 방향
 - 고유값: 고유벡터의 크기이자 해당 방향의 분산

• PCA 수행 스텝

1. 입력 데이터 세트의 공분산 행렬 생성
2. 공분산 행렬의 고유벡터와 고유값 계산
3. 고유값이 가장 큰 순으로 K개(PCA 변환 차수만큼) 고유벡터 추출
4. 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

• 사이킷런 구현 포인트

- PCA 적용 전 반드시 `StandardScaler` 로 스케일링 필요 (스케일이 다르면 분산이 큰 특성이 주성분을 독점함)

- `PCA(n_components=K)` → `fit()` → `transform()`
- `explained_variance_ratio_`: 각 주성분이 설명하는 분산 비율 확인 가능
- 적절한 차원 수 선택 방법
 - 방법1: 누적 분산 비율이 95%가 되는 최소 차원 수 계산 → `n_components=0.95`로 설정
 - 방법2: 설명된 분산 vs 차원 수 그래프에서 변곡점(Elbow) 확인

03 LDA (Linear Discriminant Analysis)

• 기본 아이디어

- PCA와 유사하게 저차원 공간에 투영해 차원을 축소하지만, 목적이 다름
- PCA: 비지도학습, 입력 데이터의 분산이 가장 큰 축을 찾음
- LDA: 지도학습, 클래스 결정 값을 사용해 클래스를 최대한 분리할 수 있는 축을 찾음

• 핵심 원리

- 클래스 간 분산(between-class scatter)을 최대화하고, 클래스 내부 분산(within-class scatter)을 최소화하는 방향으로 축을 찾음
- 즉, 클래스 분리를 최대화하는 방향 = LDA의 주성분

• LDA 수행 스텝

1. 클래스 내부 분산 행렬(S_W)과 클래스 간 분산 행렬(S_B) 계산 (입력 데이터의 결정 값 클래스별 평균 벡터 기반)
 2. $S_W^{-1} \times S_B$ 를 고유값 분해
 3. 고유값이 가장 큰 순으로 K개 고유벡터 추출
 4. 추출된 고유벡터로 입력 데이터를 새롭게 변환
- 사이킷런: `LinearDiscriminantAnalysis(n_components=K)` → `fit(X, y)` → `transform(X)` (y 레이블 필수)

04 SVD (Singular Value Decomposition)

• 기본 아이디어

- 행렬을 3개의 행렬로 분해하는 기법: $A = U\Sigma V^T$

- PCA와 유사한 행렬 분해 방식이지만, 정방행렬뿐만 아니라 행과 열의 크기가 다른 행렬에도 적용 가능
- 특이값(Singular Value): Σ 안에 들어있는 0이 아닌 대각원소들 $\rightarrow$ 데이터의 중요도를 알려주는 지표, 클수록 중요

• 구성요소

- U : 행 방향 특징 ($m \times m$ 직교행렬)
- Σ : 중요도 대각행렬 ($m \times n$, 특이값이 대각원소)
- V^T : 열 방향 특징 ($n \times n$ 직교행렬)
- U, V 에 속한 벡터는 특이 벡터(Singular Vector)로, 서로 직교하는 성질이 있음

• Truncated SVD

- Σ 의 대각원소 중 상위 P 개만 추출하고, 대응하는 U, V 의 원소도 함께 제거해 더욱 차원을 줄인 형태
- 완벽하게 복원되지는 않지만, 상당한 수준으로 원본 행렬 근사 가능 $\rightarrow$ 데이터 압축에 활용
- 사이킷런의 `TruncatedSVD`: 사이파이의 `svds`와 달리 U, Σ, V^T 를 직접 반환하지 않고, PCA처럼 `fit()`, `transform()`으로 저차원 변환 결과만 반환
- PCA와의 관계: 데이터를 스케일링(중심화)하면 SVD와 PCA는 동일한 변환 수행 (PCA가 내부적으로 SVD 알고리즘으로 구현됨)
- **PCA와의 차이**: PCA는 밀집 행렬(Dense Matrix)만 가능, SVD는 희소행렬(Sparse Matrix)도 가능

05 NMF (Non-negative Matrix Factorization)

• 기본 아이디어

- 원본 행렬을 음수를 포함하지 않는 두 개의 양수 행렬로 분해: $V \approx W \times H$
- $W(m \times p)$: 각 데이터가 잠재 특징에 얼마나 관련되는지 (특징 추출, 차원 축소에 활용)
- $H(p \times n)$: 잠재 특징이 어떤 원본 요소로 구성되는지 (토픽 모델링 등에 활용)
- p : 잠재 요소의 수 = 몇 개의 차원으로 축소할 것인지

• 증배 갱신 규칙 (Multiplicative Update Rules)

- W, H에 초기값을 무작위로 설정하고, V와 WH 간의 거리를 최소화하도록 반복적으로 값을 수정
- 곱셈 방식으로 수정하는 이유: NMF는 양수로만 계산하기 때문에 더하기/빼기 방식으로는 음수가 발생할 수 있음
- 행렬 R과 WH의 차이가 거의 0이 될 때까지 반복

• 특징

- SVD와 달리 분해된 행렬의 값이 모두 양수이므로 해석이 용이
- 텍스트의 토픽 모델링, 이미지 특징 추출 등에 활용
- 사이킷런: `NMF(n_components=K)` → `fit()` → `transform()`